

Introducción a Neo4J

En esta práctica tomaremos contacto con la base de datos de grafos de uso más extendido, **Neo4J**. Veremos la sintaxis básica del lenguaje **Cypher** y cómo cargar datos en la base de datos, tanto desde archivos **.csv** como empleando una conexión directa con una base de datos relacional.

Configuración del contenedor

Para esta base de datos vamos a usar la imagen oficial proporcionada para la versión 5.26.0. En la carpeta de software descargada del aula virtual hay dos archivos .jar. Uno de ellos es una extensión de Neo4J llamada APOC-extended, que nos dará funcionalidad adicional. El otro jar es el driver JDBC de PostgreSQL, que le permitirá a esta extensión conectarse con PostgreSQL para obtener datos directamente. Estos dos jars debes ponerlos en la carpeta que compartimos entre el host y los contenedores (en Windows c:\temp).

El primer paso es crear el contenedor a partir de la imagen:

```
docker run --name xine-grei-neo4j -p 7474:7474 -p 7687:7687 -d --platform=linux/amd64 -e  
NEO4J_AUTH=neo4j/pwalumnobd -v c:\temp:/home/alumnobd/host-temp --network network-xine-grei neo4j:5.26.0
```

Los puertos mapeados son necesarios para poder conectar con el servidor desde un navegador a través de una aplicación web que proporciona Neo4J con la instalación. La opción de entorno **NEO4J_AUTH** permite la especificación de usuario y contraseña. El nombre de usuario neo4j debemos mantenerlo, pero podemos cambiar la contraseña. Con la opción **-v** compartimos una carpeta entre el contenedor y el host. La opción **--network** añade el contenedor a la red que ya tenemos creada (si no está creada esta red tienes la instrucción para crearla en el tutorial de MongoDB).

Ahora abrimos una terminal y copiamos los archivos .jar necesarios para las extensiones APOC y APOC-extended. Después reiniciamos el contenedor.

?

```
docker exec -it xine-grei-neo4j bash
cp /home/alumnobd/host-temp/*.jar /var/lib/neo4j/plugins
cp /var/lib/neo4j/labs/apoc-5.26.0-core.jar /var/lib/neo4j/plugins
chown neo4j:adm /var/lib/neo4j/plugins/*.jar
docker restart xine-grei-neo4j
```

La shell está accesible con el comando `cypher-shell` dentro del contenedor. Además, podemos acceder a la URL <http://localhost:7474/browser/> desde el host para utilizar el cliente web proporcionado por la base de datos.

En el sistema ya tenemos creadas dos bases de datos, una base de datos *system* que almacena metadatos del sistema y una base de datos neo4j para el usuario neo4j, que usaremos para almacenar nuestros datos. La versión instalada de Neo4J no permite crear nuevas bases de datos, por lo que no debemos borrar tampoco la base de datos neo4j.

Para ver las bases de datos disponibles podemos usar el comando `show databases` en cypher-shell.

Gestión de datos con el lenguaje Cypher

El material que usamos aquí está basado en la página del manual de [introducción a Cypher](#). Más detalles pueden consultarse en el [manual de Cypher](#).

Con la cláusula create podemos añadir elementos al grafo usando patrones. La respuesta de cypher es el número de elementos creados. Con este comando creamos un nodo etiquetado como "Movie", con valores para las propiedades "title" y "released".

```
CREATE (:Movie { title:"The Matrix",released:1997 });
```

Podemos añadir una cláusula return para devolver los valores de las variables que se usan durante la creación. En este caso vemos el contenido de la variable p, es decir,

```
CREATE (p:Person { name:"Keanu Reeves", born:1964 })
CREATE (p:Person {name:"Halle Berry"})
RETURN p;
```

Se pueden crear varios elementos dentro de la misma cláusula CREATE y también usar varias cláusulas CREATE en la misma sentencia. En la siguiente sentencia estaríamos creando tres nodos y dos relaciones.

```
CREATE (a:Person { name:"Tom Hanks", born:1956 })
  -[r:ACTED_IN { roles: ["Forrest"]}]>
    (m:Movie { title:"Forrest Gump",released:1994 })
CREATE (d:Person { name:"Robert Zemeckis", born:1951 })-[:DIRECTED]->(m)
RETURN a,d,r,m;
```

Muy habitualmente, necesitamos enlazar elementos que estamos creando con elementos que ya están en la base de datos, para eso necesitaremos encajar patrones existentes en la base de datos usando la cláusula **MATCH**. En el ejemplo siguiente, creamos una nueva película y la enlazamos con el nodo de "Tom Hanks" que ya había sido creado antes.

```
match (p:Person { name:"Keanu Reeves"})
match (m:Movie { title:"The Matrix"})
create (p)-[r:ACTED_IN { roles: ['Neo']}]>(m);

MATCH (p:Person { name:"Tom Hanks" })
CREATE (m:Movie { title:"Cloud Atlas",released:2012 })
CREATE (p)-[r:ACTED_IN { roles: ['Dr. Henry Goose', 'Hotel Manager', 'Isaac Sachs',
                                'Dermot Hoggins', 'Cavendish', 'Look-a-Like Actor','Zachry']}]>(m)
RETURN p,r,m;

MATCH (p:Person {name:"Halle Berry"})
MATCH (m:Movie { title:"Cloud Atlas"})
CREATE (p)-[r:ACTED_IN {roles:['Native Woman' , 'Jocasta Ayrs', 'Luisa Rey',
                                'Indian Party Guest', 'Ovid', 'Meronym']}]>(m)
```

Las cláusulas **CREATE** se ejecutan una vez por cada fila de valores de las variables encajadas en el patrón **MATCH**. Ahora podemos comprobar que "Tom Hanks" está ya asociado a dos películas.

```
match (:Person {name:"Tom Hanks"}) -[r:ACTED_IN]-> (m:Movie)
return m.title, r.roles;
```

La cláusula **MERGE** funciona como un **MATCH** y un **CREATE** que podría ejecutarse un número ilimitado de veces y que dejaría la base de datos en el mismo estado. La cláusula intenta hacer el **MATCH**, y si no lo encuentra entonces hace el **CREATE**. Permite especificar propiedades adicionales que se modificarán después de la creación. En este caso, si no existe una película con título "Cloud Atlas", se crea, y durante la creación se asigna el valor 2012 al atributo "released".

```
MERGE (m:Movie { title:"Cloud Atlas" })
ON CREATE SET m.released = 2012
RETURN m;
```

Si el patrón que colocamos en MERGE existe solo parcialmente, se creará de nuevo entero, obteniendo datos duplicados. Si necesitamos modificar propiedades directamente podemos usar **SET** después de **MATCH**. Por ejemplo, para cambiar la propiedad "released" a 2013.

```
MATCH (m:Movie { title:"Cloud Atlas" })
SET m.released = 2013
RETURN m;
```

Para borrar elementos (nodos o relaciones) usamos la cláusula **DELETE** después de haberlos localizado con **MATCH**. Si intentamos borrar un nodo que tiene relaciones, obtendremos un error. Para forzar el borrado en cascada de relaciones al borrar nodos podemos usar la cláusula **DETACH DELETE**. También se pueden eliminar propiedades usando **REMOVE**.

```
MATCH (m:Movie { title:"Cloud Atlas" })
REMOVE m.released
RETURN m;

MATCH (m:Movie { title:"Cloud Atlas" })
DELETE m;

MATCH (m:Movie { title:"Cloud Atlas" })
DETACH DELETE m;

MATCH (p:Person { name:"Tom Hanks" })
CREATE (m:Movie { title:"Cloud Atlas",released:2012 })
CREATE (p)-[r:ACTED_IN { roles: ['Zachry']}]>(m)
RETURN p,r,m;
```

La cláusula **MATCH** permite filtrar elementos, pero solo utilizando predicados de igualdad. Para establecer otro tipo de predicados se puede utilizar la cláusula **WHERE**.

```
MATCH (m:Movie)
WHERE m.title = "Forrest Gump"
RETURN m;
```

Ojo con como funciona esta clausula. En la siguiente consulta la misma persona no puede ser p1 y p2 en la misma fila del resultado.

```
MATCH (p1:Person)-[:ACTED_IN]->(m:Movie{title:"Cloud Atlas"})<-[:ACTED_IN]-(p2:Person)
RETURN p1, p2
```

También se pueden filtrar por la etiqueta en la cláusula **WHERE**.

```
MATCH (n)
WHERE n:Person OR n:Movie
return n;
```

En el siguiente ejemplo, vemos como combinar varios predicados y como usar distintos operadores, en este caso el operador **=~** para utilizar expresiones regulares ([sintaxis de expresiones regulares de java](#)), y el operador **IN** para comprobar si un elemento está contenido en una lista (o array).

```
MATCH (p:Person)-[r:ACTED_IN]->(m:Movie)
WHERE p.name =~ "T.+" OR m.released > 2000 OR "Forrest" IN r.roles
RETURN p,r,m
```

También se pueden usar patrones en la cláusula WHERE. En este caso se buscan personas que actuaron en alguna película, pero que no dirigieron ninguna película.

```
MATCH (p:Person)-[:ACTED_IN]->(m)
WHERE NOT (p)-[:DIRECTED]->(m)
RETURN p,m
```

De esta forma podemos obtener por ejemplo los nodos que no están relacionados con ningún nodo.

```
match (n)
where not (n)---()
return n;
```

Como ya hemos visto en los ejemplos anteriores, las sentencias de Cypher devuelven filas utilizando una cláusula **RETURN**. En este ejemplo, vemos como se pueden usar literales de tipos numéricos, cadenas de caracteres, arrays y maps, para construir una fila.

```
return 3 as id,
      "ventas" as departamento,
      [{nombre:"pepe", edad:23},
       {nombre:"juan", edad:45},
       {nombre:"Ernesto", idade:45}] as empleados;
```

Para acceder a las propiedades se puede usar la sintaxis **elemento.propiedad**. También podemos usar índices para acceder a elementos concretos de los arrays.

```
match (:Person)-[r:ACTED_IN]->(:Movie)
return r.roles[0];
```

Podemos utilizar también funciones definidas en el sistema (**length(array)**, **toInteger("12")**, **coalesce(p.a,"n/a")**, ...). También podemos utilizar la palabra clave **DISTINCT** para eliminar filas duplicadas, igual que hacemos en SQL.

```
MATCH (n)
RETURN DISTINCT labels(n) AS Labels, labels(n)[0] as label, size(labels(n)) as size;
```

La agrupación y la agregación se gestionan de forma automática al combinar funciones de agregado con variables en la cláusula **RETURN**.

```
MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)<-[:DIRECTED]-(director:Person)
RETURN actor, director, count(*) AS collaborations;
```

También podemos ordenar las filas del resultado (**ORDER BY**) y controlar la paginación usando **SKIP** y **LIMIT**.

```
MATCH (a:Person)-[:ACTED_IN]->(m:Movie)
RETURN a, count(*) AS appearances
ORDER BY appearances DESC LIMIT 2;

MATCH (a:Person)-[:ACTED_IN]->(m:Movie)
RETURN a, count(*) AS appearances
ORDER BY appearances DESC SKIP 2 LIMIT 2;
```

Al igual que en SQL, podemos construir arrays usando funciones de agregado.

```
MATCH (m:Movie)<-[:ACTED_IN]-(a:Person)
RETURN m.title AS movie, collect(a.name) AS cast, count(*) AS actors;
```

Las palabras clave **UNION** y **WITH** nos permiten componer sentencias más largas a partir de otras más pequeñas. Por ejemplo, la siguiente consulta obtiene los nombres de las personas que actuaron o dirigieron una película, indicando si actuaron o dirigieron y el título de la película.

```
MATCH (actor:Person)-[:ACTED_IN]->(movie:Movie)
RETURN actor.name AS name, type(r) AS type, movie.title AS title
UNION
MATCH (director:Person)-[:DIRECTED]->(movie:Movie)
RETURN director.name AS name, type(r) AS type, movie.title AS title
```

La cláusula **WITH** se usa igual que la cláusula **RETURN**, pero solo para generar un resultado intermedio que sirva de entrada en la consulta siguiente. Por ejemplo, en la siguiente consulta nos permite filtrar en base al resultado de un agregado previamente calculado.

```
MATCH (person:Person)-[:ACTED_IN]->(m:Movie)
WITH person, count(*) AS appearances, collect(m.title) AS movies
WHERE appearances > 1
RETURN person.name, appearances, movies
```

Para terminar esta sección veremos como se pueden crear índices para acelerar las consultas y restricciones que nos permitan asegurar que los datos se ajustan a un dominio de aplicación concreto. Los índices se pueden crear en cualquier momento con una instrucción **CREATE INDEX**.

```
CREATE INDEX FOR (a:Person) ON (a.name);
```

Para ver los índices creados podemos usar el siguiente comando.

```
show indexes;
```

Para borrar un índice usamos el comando DROP INDEX. El nombre del índice que necesitamos para borrarlo lo obtenemos de la información del comando show indexes. También se puede borrar el índice por la etiqueta y propiedad usadas para crearlo.

```
drop index index_5c0607ad;  
  
drop index on :Person(name);
```

Podemos crear también restricciones de tipo UNIQUE. Cuando se crea una restricción de estas, se crea también un índice del mismo tipo.

```
CREATE CONSTRAINT ON (p:Person) ASSERT p.name IS UNIQUE  
  
show constraints;  
  
DROP CONSTRAINT constraint_e26b1a8b;
```

Importación de pequeñas cantidades de datos con el comando LOAD

En esta sección vamos a ver algunos ejemplos de uso de la cláusula **LOAD** para importar datos obtenidos en archivos **CSV**. Con esta cláusula tenemos control sobre cómo importamos los datos y cómo los integramos con el grafo existente; sin embargo, no está diseñada para importar grandes cantidades de datos. Para eso emplearemos el comando **neo4j-admin import**. Sin embargo, **neo4j-admin import** no puede usarse sobre una base de datos que ya haya sido utilizada y, dada la limitación que tiene la edición **Community** para usar varias bases de datos, no es recomendable su uso con esta edición.

Antes de comenzar emplearemos **SQL** para generar los archivos necesarios a partir de la base de datos de películas que tenemos en **PostgreSQL**. Exportaremos a **CSV** con un formato concreto los datos de las 20 películas de mayor presupuesto. Ejecutaremos estos tres comandos en una terminal del contenedor de postgresql (bdge-mbd-pgsql). Recuerda ejecutar los comandos desde la cuenta del usuario alumnobd.


```
docker exec -it xine-grei-pgsql bash
su alumnobd

cd /home/alumnobd/host-temp

psql -d xine -c "\copy
(select id, titulo, presupuesto, fecha_emision, ingresos, duracion
from peliculas
order by presupuesto desc, id
limit 20)
to '/home/alumnobd/host-temp/peliculas.csv'
with csv header"

psql -d xine -c "\copy
(select p.id as id_pelicula, e->>'id' as id_persona, e->>'name' as nombre, e->>'job' as trabajo
from (select id
      from peliculas p
      order by presupuesto desc, id
      limit 20) as p,
      creditos c, json_array_elements(c.personal) as pe(e)
where p.id = c.pelicula)
to '/home/alumnobd/host-temp/personal.csv'
with csv header"

psql -d xine -c "\copy
(select p.id as id_pelicula, e->>'id' as id_persona, e->>'name' as nombre, e->>'character' as personaje
from (select id
      from peliculas p
      order by presupuesto desc, id
      limit 20) as p,
      creditos c, json_array_elements(c.reparto) as r(e)
```

```
where p.id = c.pelicula)
to '/home/alumnobd/host-temp/reparto.csv'
with csv header"
```

Vaciamos ahora la base de datos **Neo4J**, para tener solo el grafo que vamos a importar. Limitamos el borrado a **10000 nodos** para no tener problemas en la ejecución del comando. Si después de borrar 10000 nodos aún quedan datos, repetimos el comando las veces necesarias.

```
match(n) with n limit 10000 detach delete n;
```

Para asegurarnos de que los datos se importan correctamente, y que después se pueden acceder de forma ágil, podemos definir algunas **restricciones** y **índices**.

```
CREATE CONSTRAINT idPelicula FOR (p:Pelicula) REQUIRE p.id IS UNIQUE;

CREATE CONSTRAINT idPersona FOR (p:Persona) REQUIRE p.id IS UNIQUE;

CREATE INDEX FOR (p:Pelicula) ON (p.titulo);

CREATE INDEX FOR (p:Persona) ON (p.nombre);
```

Para poder importar los datos, debemos moverlos al directorio de importación **/var/lib/neo4j/import**. Creamos ahora un nodo para cada película que exportamos en el archivo **"peliculas.csv"**

```
LOAD CSV WITH HEADERS FROM "file:///peliculas.csv" AS csvPelicula
CREATE (p:Pelicula {id: toInteger(csvPelicula.id),
    titulo: csvPelicula.titulo,
    presupuesto: toInteger(csvPelicula.presupuesto),
    fechaEmision: date(csvPelicula.fecha_emision),
    ingresos: toInteger(csvPelicula.ingresos),
    duracion: toInteger(csvPelicula.duracion)});
```

Comprobamos que los datos fueron importados con una consulta sencilla.

```
MATCH (p) RETURN (p);
```

Ahora vamos a importar los datos de los créditos de cada película, comenzando por el personal que trabajó en la película. Antes de proceder a la importación de los datos, comprobamos que las condiciones que vamos a aplicar funcionan con una consulta.

```
LOAD CSV WITH HEADERS FROM "file:///personal.csv" AS csvPersonal
MATCH (pe:Película {id:toInteger(csvPersonal.id_película)})
RETURN pe.título, csvPersonal.nombre, csvPersonal.trabajo;

CALL {
LOAD CSV WITH HEADERS FROM "file:///personal.csv" AS csvPersonal
MATCH (pe:Película {id:toInteger(csvPersonal.id_película)})
MERGE (p:Persona {id:toInteger(csvPersonal.id_persona)})
ON CREATE SET p.nombre = csvPersonal.nombre
  CREATE (p)-[:TRABAJO_EN {trabajo:csvPersonal.trabajo}]->(pe)}
IN TRANSACTIONS;
```

Comprobamos ahora con una consulta que los datos están importados.

Recupera para cada película cuyo título comience por "**Pirat**" su **id**, su **título**, el número de personas que trabajaron en la película y un **mapeo** con la lista de nombres y cargos de las personas involucradas.

```
MATCH (persona)-[:TRABAJO_EN]->(película)
WHERE película.título =~ "Pirat.+"
RETURN película.id,
       película.título,
       COUNT(*) AS numPersonas,
       COLLECT({nombre:persona.nombre,trabajo:t.trabajo}) AS personal;
```

Para finalizar, importamos los datos del **reparto** de la película, es decir, de las personas que actuaron en la película y de los personajes que interpretaron.

```
CALL {  
LOAD CSV WITH HEADERS FROM "file:///reparto.csv" AS csvReparto  
MATCH (pe:Película {id:toInteger(csvReparto.id_película)})  
MERGE (p:Persona {id:toInteger(csvReparto.id_persona)})  
ON CREATE SET p.nombre = csvReparto.nombre  
CREATE (p)-[:ACTUO_EN {personaje:csvReparto.personaje}]->(pe)  
} IN TRANSACTIONS;
```

Comprobamos que los datos están cargados con una consulta sencilla.

```
MATCH (p:Persona)-[r:ACTUO_EN]->(pe)  
WHERE r.personaje =~ ".*Jack.*"  
RETURN p.nombre, r.personaje, pe.titulo, pe.fechaEmision;
```

Consulta de datos relacionales con JDBC

Una alternativa a la importación de datos **CSV**, cuando los datos ya están almacenados en un SGBD relacional, es el uso de **JDBC** para acceder directamente al gestor y recuperar los datos.

Para poder hacer esto, debemos tener instalada la versión **EXTENDED** de **APOC** y tener el **driver JDBC** del gestor en la carpeta de plugins.

Para ilustrar esto, realizaremos las consultas necesarias para importar datos de películas directamente desde nuestra base de datos **PostgreSQL**.

Antes de empezar con la importación, debemos asegurarnos de que el contenedor de PostgreSQL está en la misma red que el contenedor de Neo4j, para que podamos acceder a la base de datos sin problema. Para conectar el contenedor de PostgreSQL a la red que estamos usando en la materia podemos usar el siguiente comando.

```
docker network connect network-xine-grei xine-grei-pgsql
```

Eliminamos antes de empezar todos los nodos y relaciones del grafo.

```
MATCH(n) WITH n LIMIT 10000 DETACH DELETE n;
```

Ahora ya podemos importar los nodos de las películas.

Empezamos generando los datos de las películas.

```
with "jdbc:postgresql://xine-grei-pgsql/xine?user=alumnobd&password=pwalumnobd" as url
CALL apoc.load.jdbc(url,
"select id, titulo, presupuesto, fecha_emision, ingresos, duracion
from peliculas order by presupuesto desc, id limit 1000") yield row
CREATE (p:Película {id:row.id,
                    titulo:row.titulo,
                    presupuesto: row.presupuesto,
                    fechaEmision:row.fecha_emision,
                    ingresos:row.ingresos,
                    duracion:row.duracion});
```

Ahora generamos los nodos de las personas.

```
with "jdbc:postgresql://xine-grei-pgsql/xine?user=alumnobd&password=pwalumnobd" as url
CALL apoc.load.jdbc(url,
"select distinct id, first_value(nombre) over (partition by id order by nombre) as nombre
from (select cast(e->>'id' as integer) as id, e->>'name' as nombre
     from credits, json_array_elements(personal) as p(e)
     where pelicula in (select id from peliculas order by presupuesto desc, id limit 1000)
     union all
     select cast(e->>'id' as integer) as id, e->>'name' as nombre
     from credits, json_array_elements(reparto) as r(e)
     where pelicula in (select id from peliculas order by presupuesto desc, id limit 1000)) as t") yield row
CREATE (p:Persona {id:row.id,
                  nombre:row.nombre});
```

Para acelerar la inserción de las relaciones, añadimos primero índices sobre las Películas y Personas, para que su búsqueda en cada inserción se realice más rápido. Vamos a añadir también índices sobre el nombre de la persona y sobre el Título de la película para acelerar consultas sobre estos campos.

```
CREATE INDEX FOR (a:Persona) ON (a.id);
CREATE INDEX FOR (a:Película) ON (a.id);
CREATE INDEX FOR (a:Persona) ON (a.nombre);
CREATE INDEX FOR (a:Película) ON (a.título);
```

Ahora ya podemos proceder a insertar las relaciones que definen que persona ha actuado en que película.

```
with "jdbc:postgresql://xine-grei-pgsql/xine?user=alumnobd&password=pwalumnobd" as url
CALL apoc.load.jdbc(url,
"select cast(e->>'id' as integer) as persona, e->>'character' as personaje, película
from credits, json_array_elements(reparto) r(e)
where película in (select id from películas order by presupuesto desc, id limit 1000)") yield row
MATCH (película:Película {id:row.película})
MATCH (persona:Persona {id:row.persona})
CREATE (persona)-[r:ACTUO_EN {personaje:row.personaje}]->(película);
```

Finalmente procedemos a insertar las relaciones que definen que persona ha trabajado en que película.

```
with "jdbc:postgresql://xine-grei-pgsql/xine?user=alumnobd&password=pwalumnobd" as url
CALL apoc.load.jdbc(url,
"select cast(e->>'id' as integer) as persona, e->>'job' as trabajo, película
from credits, json_array_elements(personal) p(e)
where película in (select id from películas order by presupuesto desc, id limit 1000)") yield row
MATCH (película:Película {id:row.película})
MATCH (persona:Persona {id:row.persona})
CREATE (persona)-[r:TRABAJO_EN {trabajo:row.trabajo}]->(película);
```

Ahora ya podemos realizar consultas que exploten las relaciones entre personas y películas. Vamos a ver en primer lugar como usamos [patrones de tamaño variable](#).

Obtener el número de personas que han colaborado de forma directa con "Penélope Cruz", es decir, que han actuado o trabajado en la misma película.

```
match (penelope:Persona{nombre: "Penélope Cruz"})-->()<--(p)
return count(distinct p.id);
```

Obtener el número de personas que han actuado con alguna persona con la que haya actuado "Penélope Cruz" (relaciones de segundo nivel de tipo ACTUO_EN)

```
match (penelope:Persona{nombre: "Penélope Cruz"})-[:ACTUO_EN*..4]-(p:Persona)
return count(distinct p.id);
```

Obtener el número de personas que ha colaborado con alguna persona con la que haya colaborado "Penélope Cruz" (relaciones de segundo nivel de cualquier tipo).

```
match (penelope:Persona{nombre: "Penélope Cruz"})-[*..4]-(p:Persona)
return count(distinct p.id);
```

Para la siguiente consulta necesitamos calcular el camino más corto en un par de nodos.

Obtener el camino más corto entre "Kevin Bacon" y "Penélope Cruz" a través de relaciones tipo ACTUO_EN. Obtén el camino más corto usando cualquier tipo de relación también.

```
MATCH (k:Persona {nombre: 'Kevin Bacon'} ),
      (a:Persona {nombre: 'Penélope Cruz'}),
      p = shortestPath((k)-[:ACTUO_EN*]-(a))
RETURN length(p), p;
```

```
MATCH (k:Persona {nombre: 'Kevin Bacon'} ),
      (a:Persona {nombre: 'Penélope Cruz'}),
      p = shortestPath((k)-[*]-(a))
RETURN length(p), p;
```

Última modificación: luns, 9 de febreiro de 2026, 19:26